

Entrega 2: Árboles y Jerarquías

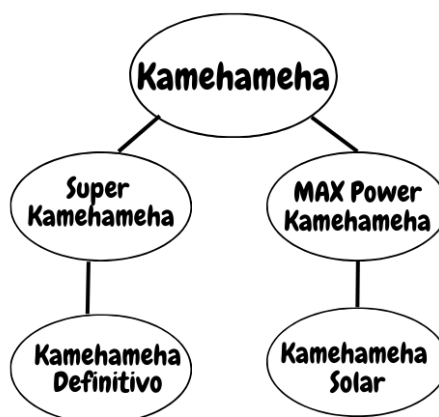
- *Explicar la diferencia entre árboles binarios y árboles generales. Ejemplificar con jerarquías de personajes y evolución de habilidades.*
- *Analizar la eficiencia de los recorridos y búsquedas en cada tipo de árbol.*

Los **árboles generales** al permitir que cada nodo tenga cualquier cantidad de hijos, hacen que sea ideal para representar jerarquías complejas con muchas ramas y niveles.

Nosotros particularmente lo usamos para modelar las habilidades y sus mejoras, mostrando evoluciones en ellas. Por ejemplo: la habilidad base *Kamehameha* tiene mejoras posibles.

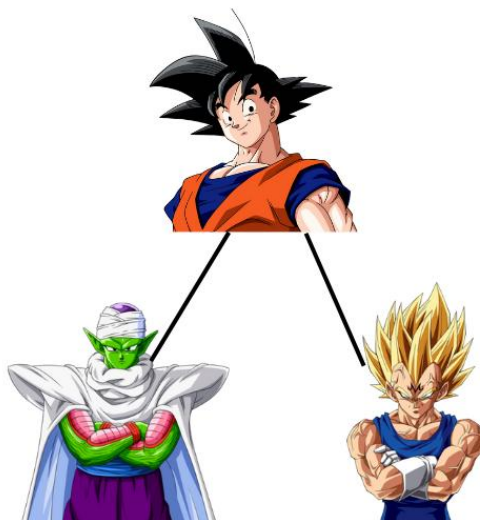
Otros ejemplos son las *técnicas de Ki*, las *transformaciones de Goku*, entre otros.

Pero de haber querido utilizar **árboles binarios** para el mismo modelado, no habría sido posible ya que estos árboles solo permiten dos hijos por nodo (hijo izquierdo e hijo derecho).



En nuestro trabajo utilizamos los árboles binarios para ordenar personajes según su nivel de poder total, comparando sus valores y ubicándolos a la izquierda o derecha según correspondiese. Por ejemplo, en un combate según los niveles de poder se acomodarían quizá de esta forma los personajes:

- Si Vegeta tiene menos poder que Goku → va a la izquierda
- Si Piccolo tiene menos poder que Vegeta → va a la izquierda de Vegeta.



Trabajo Final para la Cátedra: Estructuras de Datos en Python

Integrantes: Álvarez Rocío, Varela Ian, Gómez Morena.

Respecto a la eficiencia, los *árboles generales* presentan una complejidad $O(n)$ para recorridos y búsquedas, debido a que un nodo puede tener múltiples hijos y no existe un orden que permita descartar ramas, por lo que se recorre linealmente toda la estructura.

Por ejemplo: el árbol de habilidades de Goku, cada técnica puede tener varias mejoras. Como no hay un orden fijo, para encontrar una habilidad específica, como 'God Kamehameha', puede ser necesario recorrer todas las ramas.

En tanto en nuestros árboles binarios:

- Si el árbol está balanceado:
 - Búsqueda: **$O(\log n)$**
 - Inserción: **$O(\log n)$**
 - Recorrido inorden: **$O(n)$**
- Si el árbol está desbalanceado:
 - Búsqueda: **$O(n)$**
 - Inserción: **$O(n)$**

Por ejemplo, al buscar al personaje más fuerte:

- Si el árbol está balanceado → lo encontramos más rápido
- Si está desbalanceado → hay que recorrer mucho más

